MULTI-MODAL AI · CYBERSECURITY · MACHINE LEARNING

Multi-Modal Phishing Intelligence and Defense

Detecting malicious URLs using a soft-voting AI ensemble combining structural features (XGBoost), linguistic semantics (MiniLM), and visual heuristics.

PROJECT PRESENTATION

BY- Bibek Dhakal

Project Objective & Problem Statement

The Problem

- Phishing websites have become increasingly sophisticated and can easily bypass traditional security methods such as blacklists and signature-based detection.
- Most existing solutions rely on only one type of analysis, which limits their ability to detect modern phishing attacks.
- This project develops a multi-modal phishing detection system that combines structural, linguistic, and visual intelligence to classify URLs as Legitimate, Suspicious, or Phishing.

TARGET CATEGORIES

- **Tabular: 79 structural and mathematical URL features extracted for machine learning.**
- **NLP: Analyzes URL text patterns, semantic structure, and typosquatting attempts.**
- **Vision (MOCKED): Designed to identify brand spoofing and suspicious web page layouts.**
- **Fusion: Combines predictions using a weighted soft-voting ensemble.**

Dataset Collection & Preprocessing

35,000+
ISCX 2016 Dataset

35,000+ URL data used for training the tabular model.

235,000+
PhiUSIIL Dataset

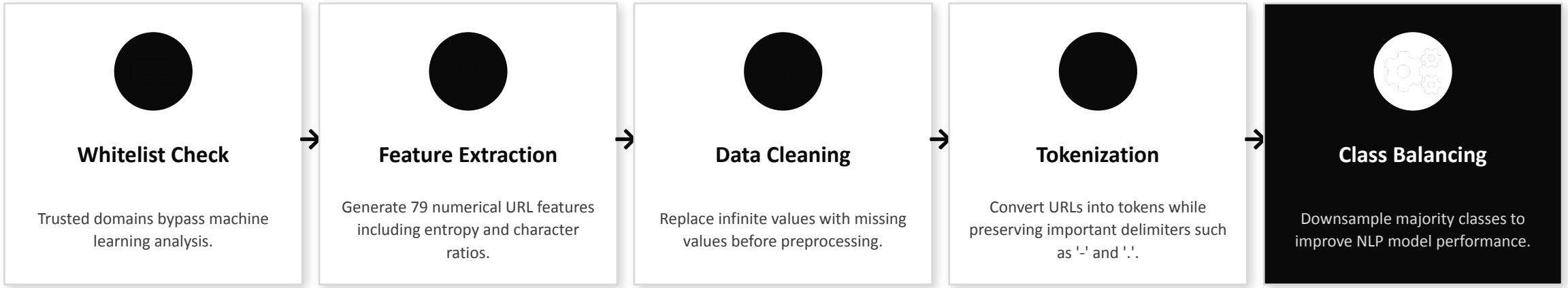
235,000+ URLs used for training the NLP model.

79
79 Features

Structural and statistical URL features extracted offline.

40/40/20
Soft Voting
40% XGBoost
40% MiniLM
20% Vision (Static MOCK)

PREPROCESSING PIPELINE



Data Analysis & Visualization

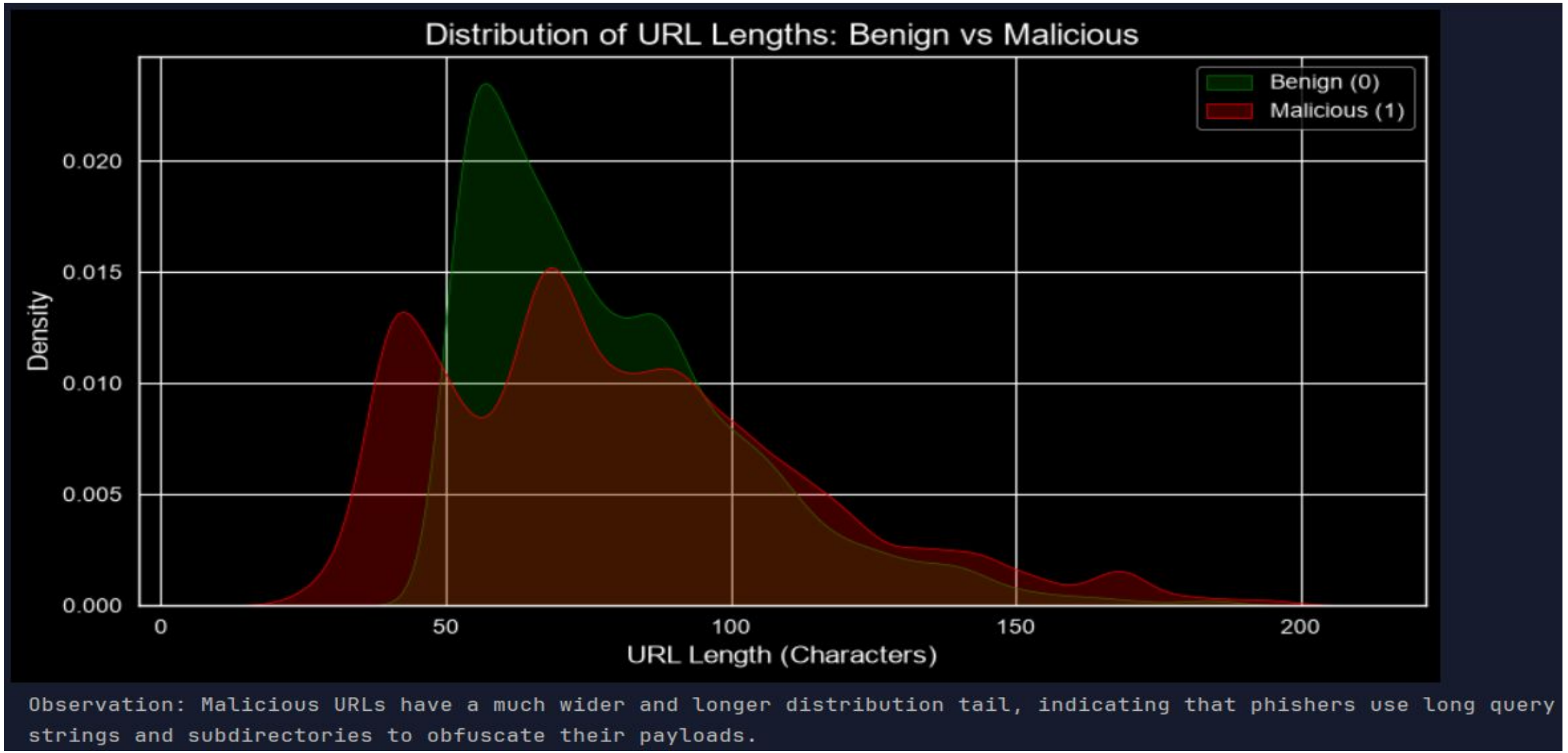
CATEGORY DISTRIBUTION (POST-FILTER)

As you can see in the Target Distribution chart, our raw dataset had a significant class imbalance, heavily skewed towards malicious URLs. To prevent model bias, especially in the NLP transformer, I implemented class balancing techniques (downsampling) during preprocessing.

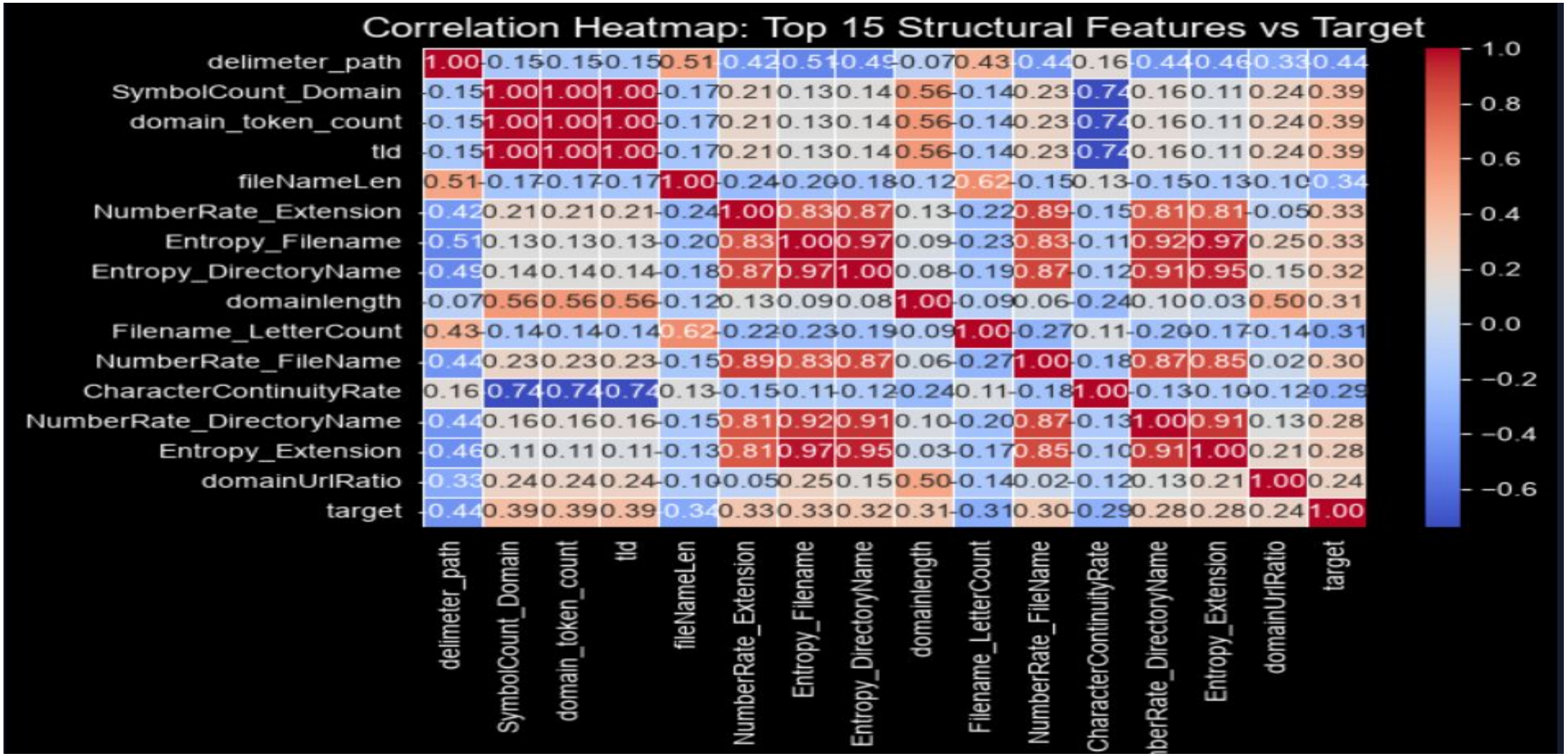
KEY OBSERVATIONS

- Feature engineering sometimes creates infinite values during ratio calculations. These values are converted to missing values before model training.
- Special characters such as hyphens and dots are preserved because they provide important phishing indicators.
- XGBoost probabilities are calibrated to produce reliable confidence scores before ensemble voting.
- MiniLM was selected because it provides strong language understanding while requiring significantly less GPU memory than larger transformer models.

For the NLP pipeline, we needed to optimize VRAM usage. I plotted the distribution of URL token lengths and found that a truncation limit of 128 tokens safely captures over 95% of URL semantics. This saved massive computational resources without sacrificing accuracy.



Before training, I conducted extensive Exploratory Data Analysis to validate why we extract 79 structural features. The density plot clearly shows that malicious URLs have a much wider and longer length distribution as attackers use long query strings to obfuscate payloads.

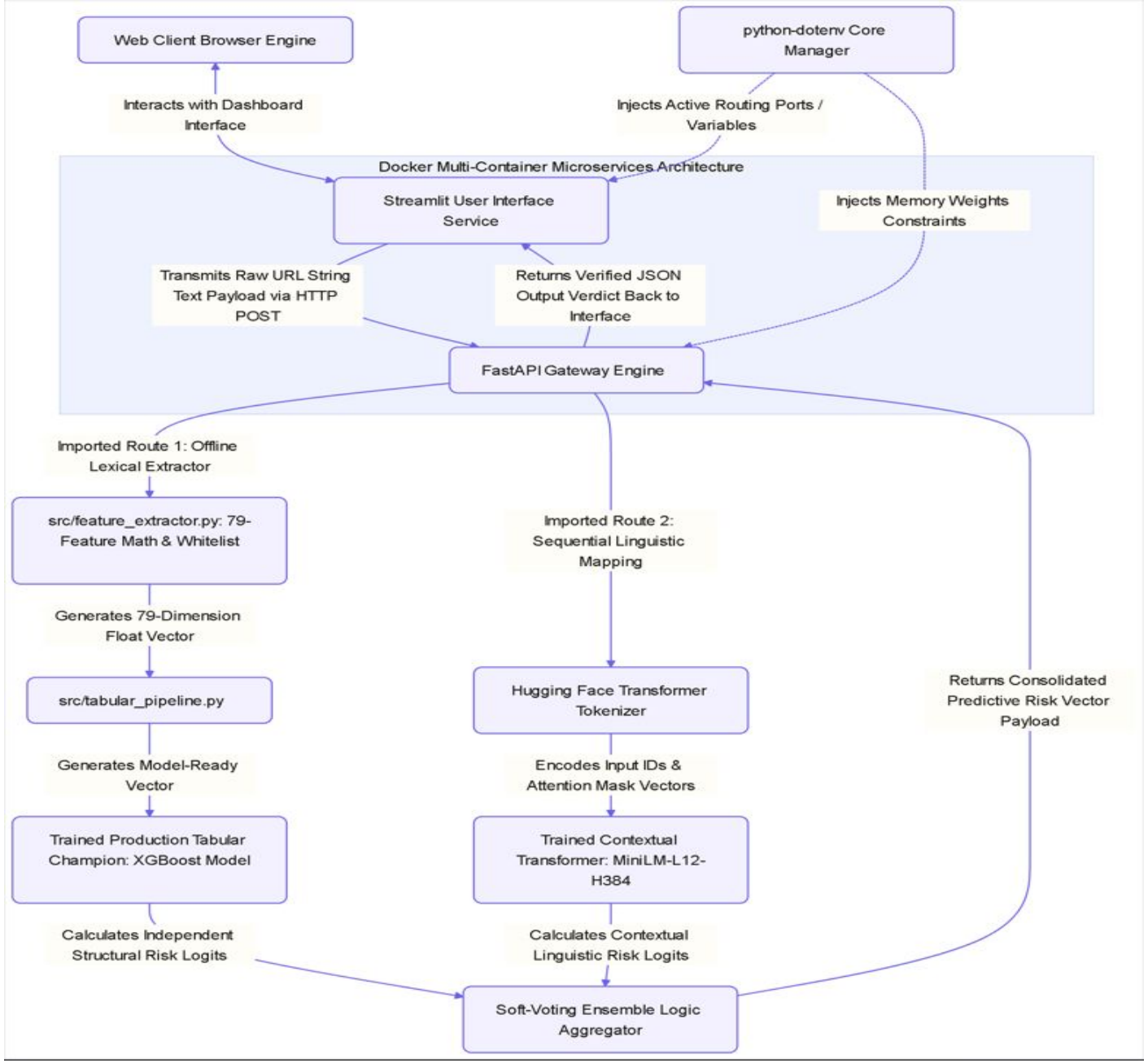


Furthermore, the correlation heatmap proves that specific engineered features—like directory entropy and delimiter counts—have a strong mathematical correlation with the phishing target variable, proving our feature extraction pipeline works.

Architecture & Design Decisions

Key Engineering Observations

- Handling Infinity: URL ratios can produce inf; converted to NaN before imputation.
- Delimiter Preservation: Kept - and . to improve typosquatting detection.
- Model Calibration: Applied Platt scaling (CalibratedClassifierCV) to calibrate XGBoost probabilities.
- VRAM Constraints: Used MiniLM-L12 instead of BERT for efficient 6GB GPU inference.
- Extensible Fusion: Vision branch is currently mocked (0.05), preserving the 3-model ensemble for future CNN integration.



Machine Learning Algorithms & Model Development

Structural features

79 numerical features extracted from each URL.

Tokenized URLs

T

Transformer input IDs and attention masks generated using MiniLM.

ALGORITHMS COMPARED (11 MODELS)

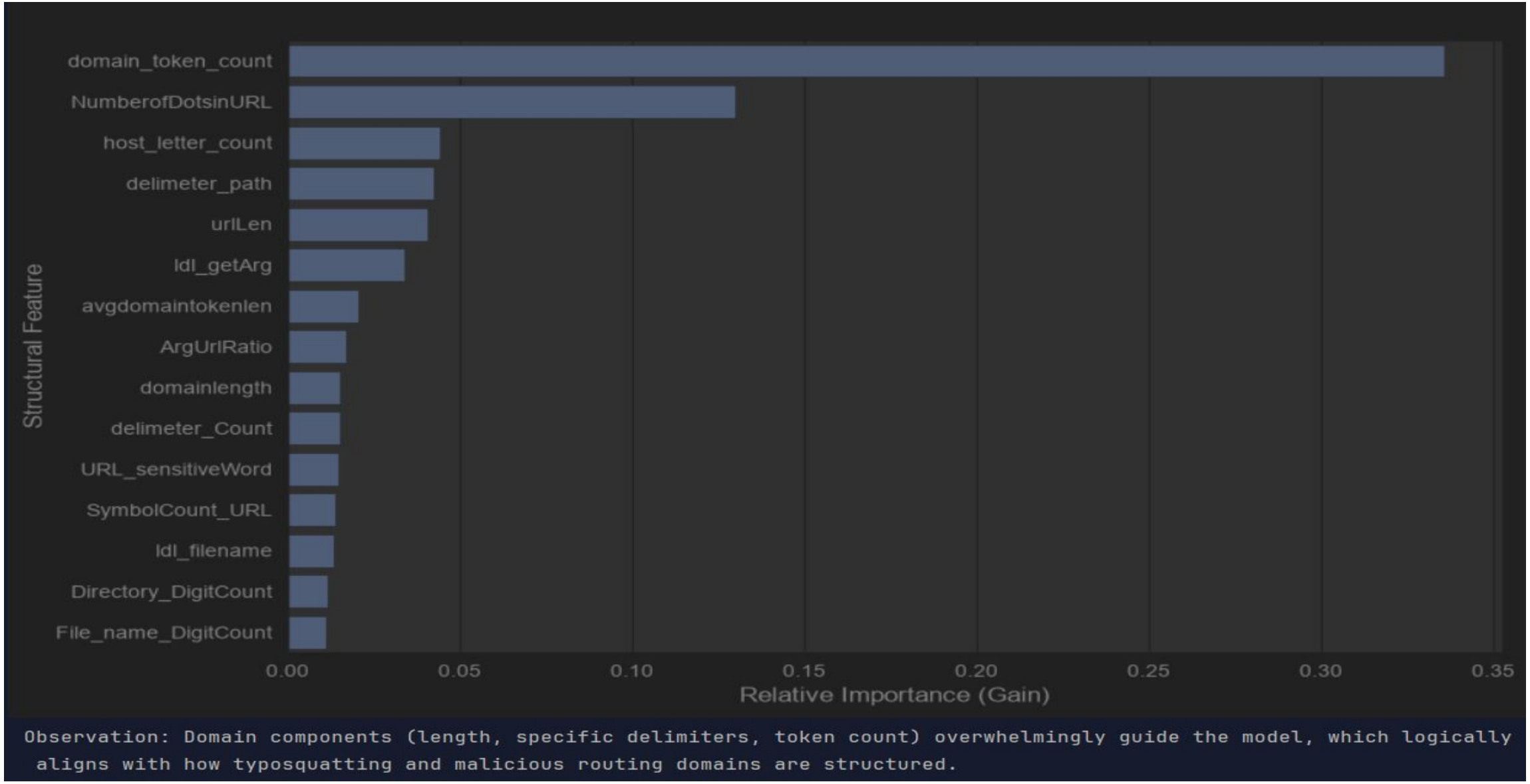
Logistic regression	Naive Bayes	K-Nearest Neighbors	Support Vector Machine
Decision Tree	Random Forest	AdaBoost	Artificial Neural network
XGBoost			

NLP

MiniLM-L12

Selection Process

- Nine traditional machine learning algorithms were evaluated using the ISCX dataset.
- XGBoost achieved the best overall performance for structural feature classification.
- MiniLM was selected for natural language analysis because it effectively captures contextual URL patterns.
- The final prediction is produced by combining both models through a weighted soft-voting ensemble.



Out of the 9 classical algorithms I tested, XGBoost emerged as the champion. A key reason is how it handles feature importance. As this chart shows, the model intelligently prioritized features like domain_token_count and NumberofDotsinURL—which aligns perfectly with human cybersecurity heuristics—to make its splits.

Model Evaluation & Results

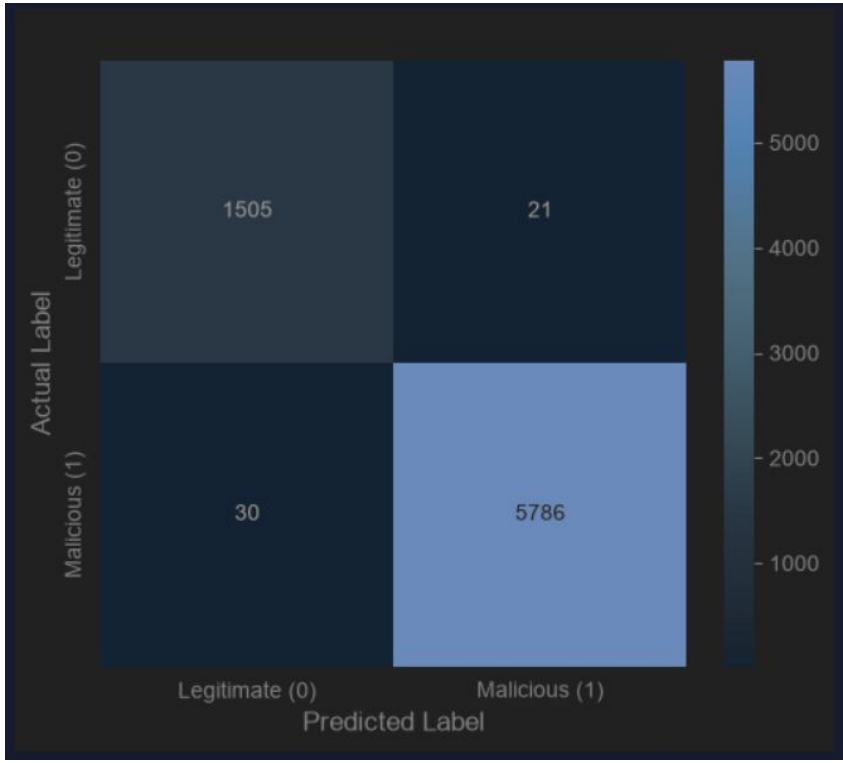
BEST CONFIGURATION

Tabular (XGBoost) + NLP (MiniLM-L12)

Soft-Voting Ensemble (40/40/20)

UNIFIED RISK METRIC

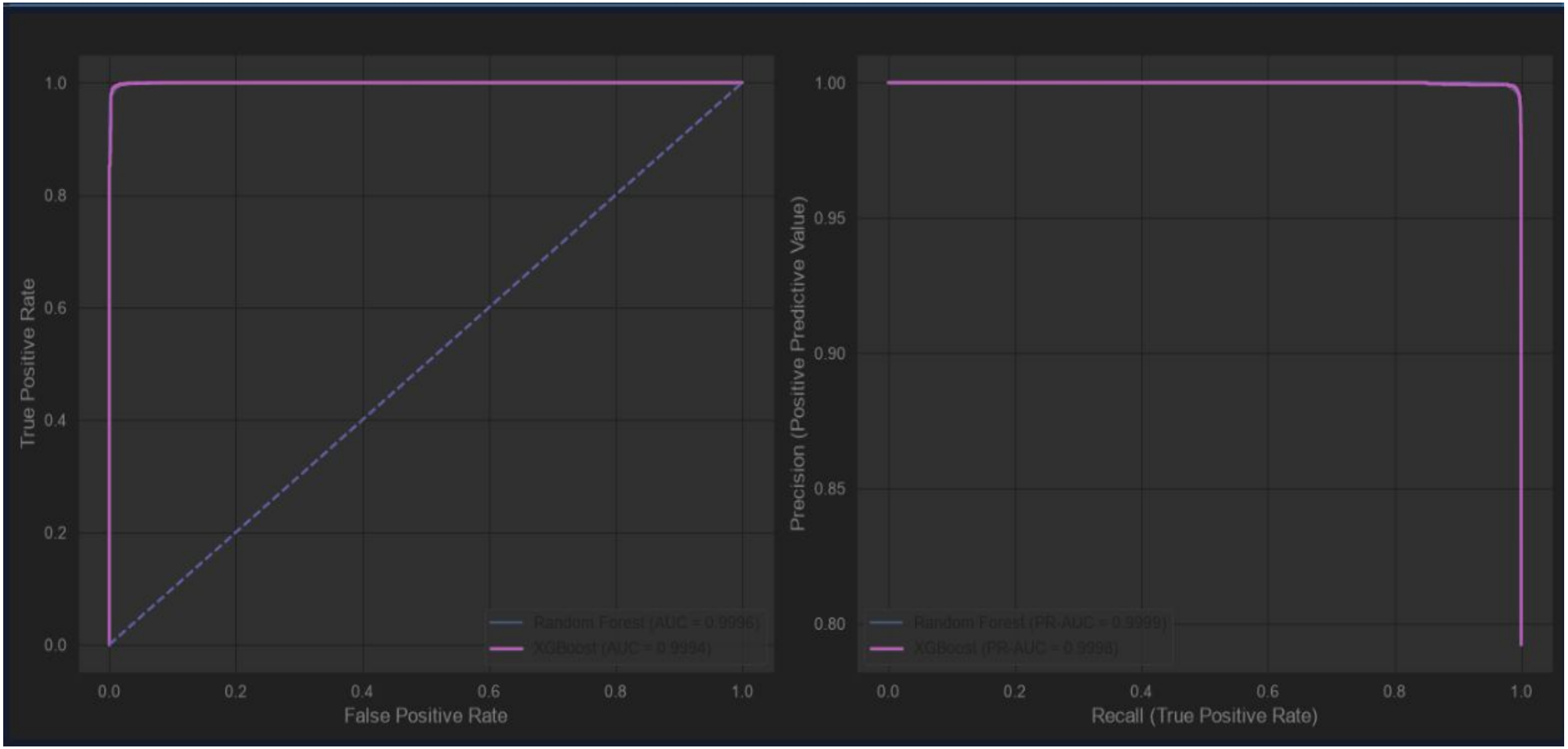
(Threshold: ≥ 0.75 = Phishing)



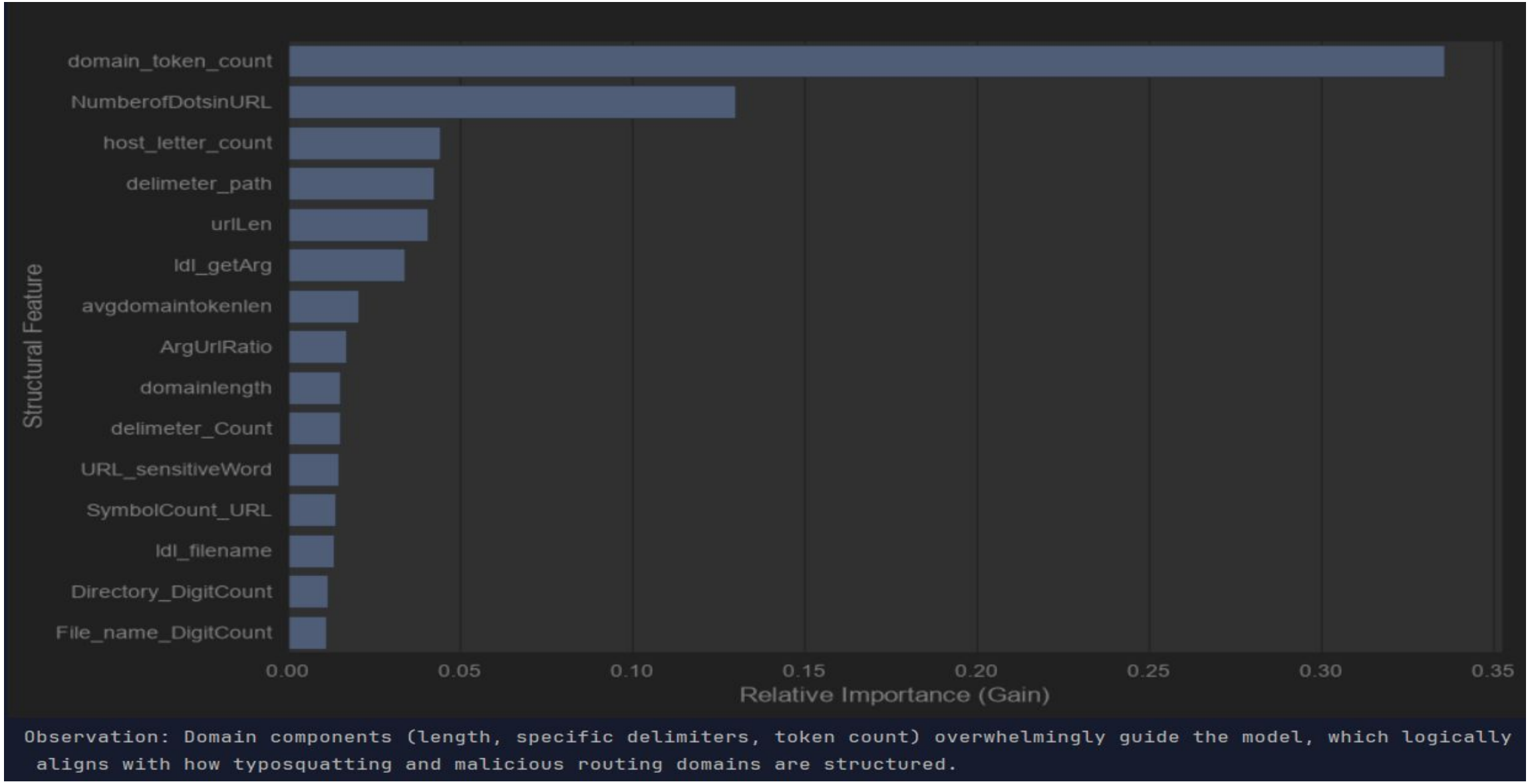
--- XGBoost Classification Report ---

	precision	recall	f1-score	support
Legitimate	0.9805	0.9862	0.9833	1526
Malicious	0.9964	0.9948	0.9956	5816
accuracy			0.9931	7342
macro avg	0.9884	0.9905	0.9895	7342
weighted avg	0.9931	0.9931	0.9931	7342

This slide highlights the standalone performance of the Tabular XGBoost branch. I used ROC-AUC to compare models because of the initial class imbalance. XGBoost achieved a near-perfect AUC of 0.9994.



This slide highlights the standalone performance of the Tabular XGBoost branch. I used ROC-AUC to compare models because of the initial class imbalance. XGBoost achieved a near-perfect AUC of 0.9994.



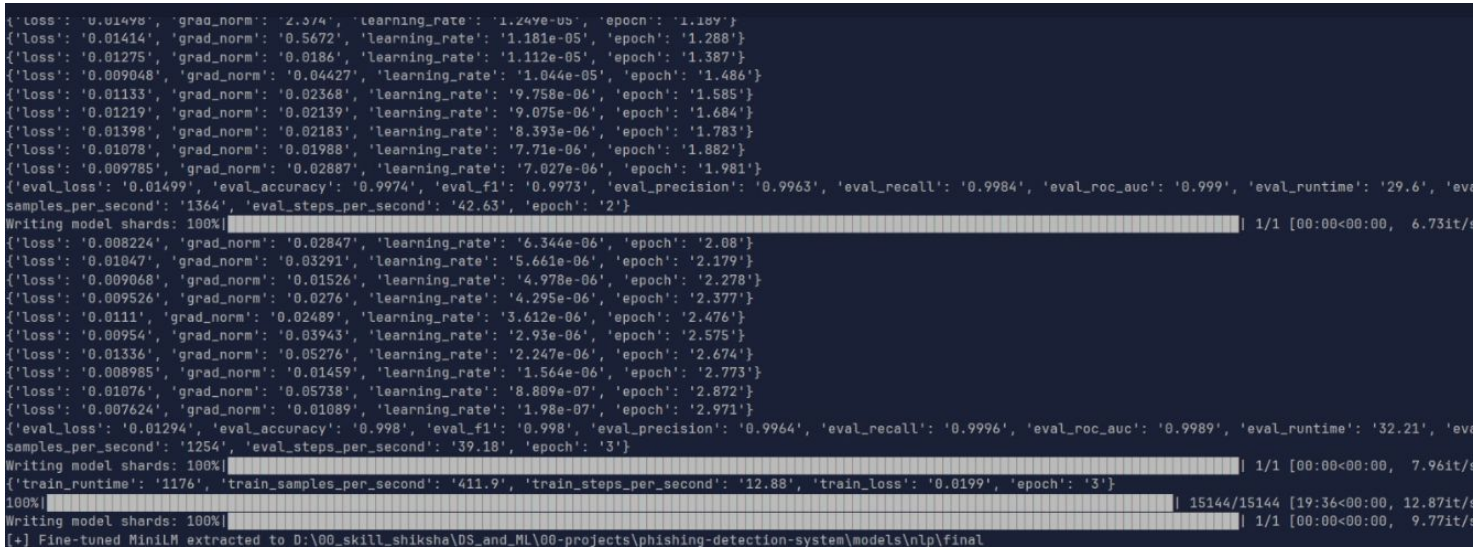
Out of the 9 classical algorithms I tested, XGBoost emerged as the champion. A key reason is how it handles feature importance. As this chart shows, the model intelligently prioritized features like domain_token_count and NumberofDotsinURL—which aligns perfectly with human cybersecurity heuristics—to make its splits.

```
[*] Loading ISCX dataset from D:\00_skill_shiksha\DS_and_ML\00-projects\phishing-detection-system\data\iscx_phishing\All.csv

[*] Training and Evaluating Mathematical/Structural Models...
----------------------------------------------------------------
Logistic Regression  | Acc: 0.9278 | F1: 0.9544 | AUC: 0.9736
Naive Bayes          | Acc: 0.5885 | F1: 0.6523 | AUC: 0.9174
KNN                  | Acc: 0.9873 | F1: 0.9920 | AUC: 0.9954
SVM                  | Acc: 0.9684 | F1: 0.9800 | AUC: 0.9926
Decision Tree        | Acc: 0.9860 | F1: 0.9911 | AUC: 0.9805
Random Forest        | Acc: 0.9925 | F1: 0.9953 | AUC: 0.9996
AdaBoost             | Acc: 0.9239 | F1: 0.9520 | AUC: 0.9752
ANN (MLP)            | Acc: 0.9892 | F1: 0.9932 | AUC: 0.9979
XGBoost              | Acc: 0.9941 | F1: 0.9963 | AUC: 0.9996

[+] Production Champion (XGBoost) saved to D:\00_skill_shiksha\DS_and_ML\00-projects\phishing-detection-system\models\tabular\xgboost_production.pkl
```

Tabular Ml: Training and Benchmark of ML models on ISCX URL dataset



NLP: Fine Tuning MiniLM-L12 on Phi USIL URL dataset

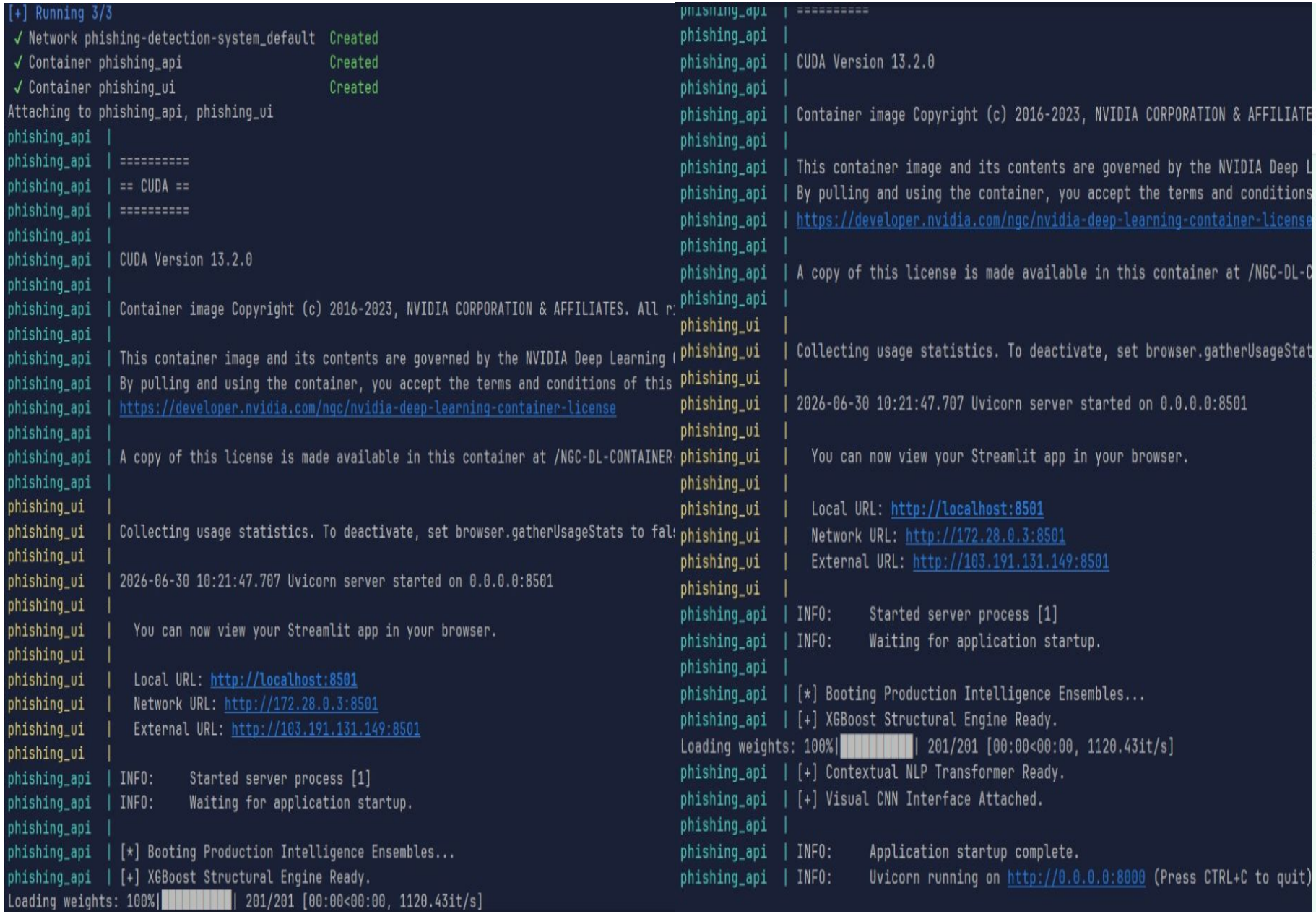
System Deployment & Aggregation

BEST CONFIGURATION

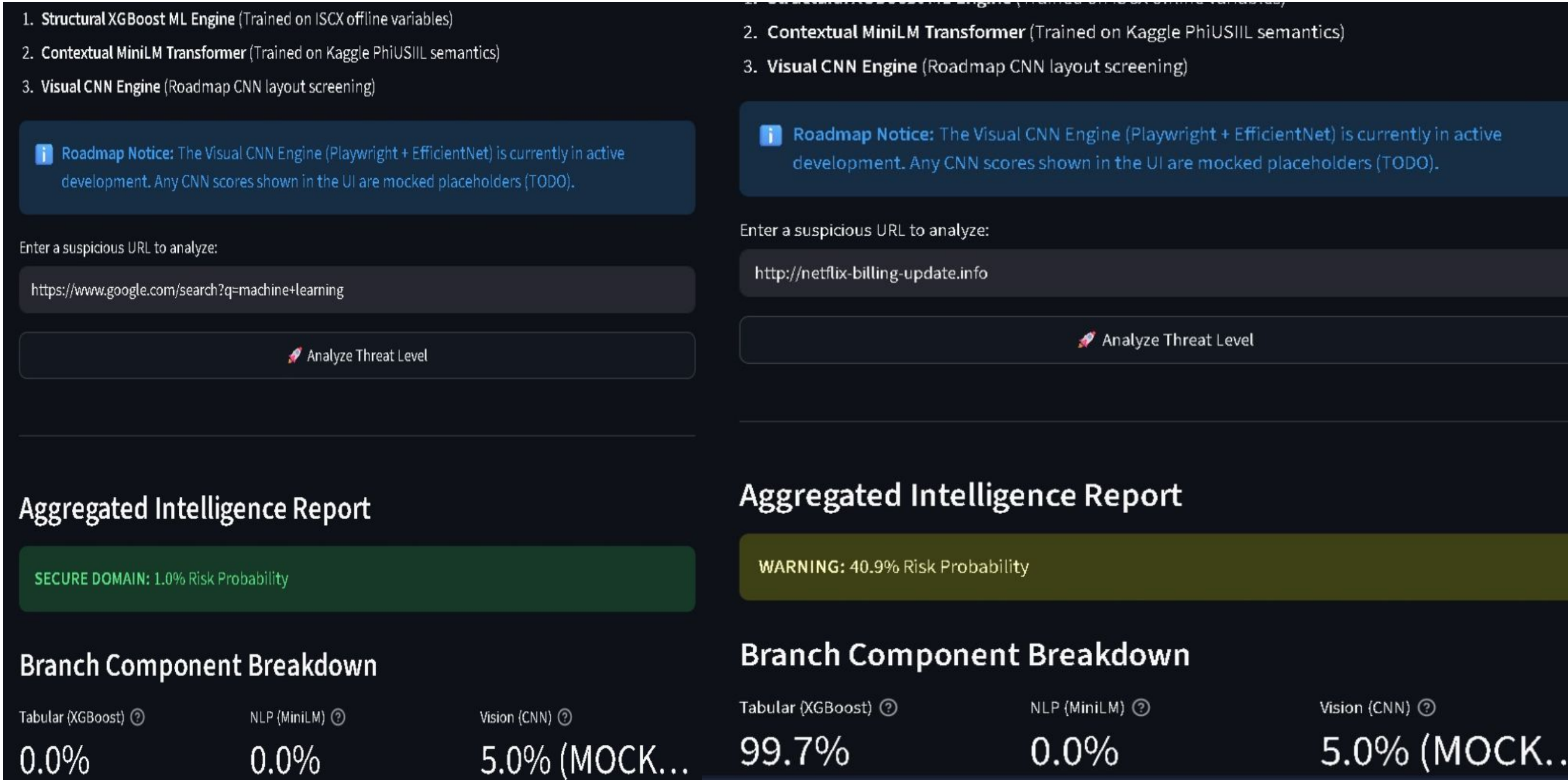
Tabular (XGBoost) + NLP (MiniLM-L12)

Deployment Architecture

- **Decoupled Microservices: Streamlit UI container communicates via HTTP POST to a heavy FastAPI backend container.**
- **Zero-Latency Loading: Models are loaded into memory asynchronously at startup via FastAPI lifespan to prevent disk-read delays.**
- **GPU Passthrough: API runs on a massive NVIDIA CUDA 13.2 base image, caching PyTorch in a separate Docker layer for ultra-fast rebuilds.**
- **Graceful Degradation: If a model branch fails, the try/except logic degrades its score to 0.0 instead of crashing the server.**



Dockerization Logs and FastAPI Swagger Test



Streamlit UI

Tools & Technologies Used

Python

Core programming language used for the entire project.

Pandas & NumPy

Data loading, cleaning, and manipulation

Scikit-learn & XGBoost

Machine learning, preprocessing, and probability calibration. XGBoost as Gradient boosting classifier.

Pytorch & Hugging Face Transformers

Deep learning framework, AutoTokenizer, and MiniLM-L12 sequence classification.

FastAPI & Pickle

High Performance asynchronous backend gateway and serialized model registry loading.

Streamlit, Docker & CUDA

Decoupled frontend UI, Containerization and NVIDIA GPU pass through for inference.

Key Findings, Challenges & Future Work

Key Findings

- Fusing multiple modalities prevents single-point bypasses by attackers.
- Calibrating XGBoost ensures mathematical compatibility with NLP Softmax outputs.
- Offline structural extraction (79 features) is drastically faster than live WHOIS network calls.

Challenges

- Managing Docker layer caching for massive PyTorch dependencies.
- Preventing Scikit-learn crashes due to division-by-zero (inf) URL feature anomalies.
- Balancing NLP classes via downsampling caused some information loss.

Future Improvements

- The current Vision mock adds a static 0.01 offset to all scores (0.05 score × 0.20 weight). Future work will replace this with a live Playwright/EfficientNet image classifier.
- Create a unified hold-out test set to mathematically tune the 40/40/20 ensemble weights.
- Migrate the static 12-domain whitelist to a live Tranco top 1M database.

Thank You

Multi-Modal Phishing Intelligence & Defense Platform

Questions & Discussion